

Order Tracking of Mobius Propeller Data

The Matlab code for this problem is listed in Appendix 1.

Trigger Signal and Representative Pressure Recording

Figure 1 shows the angular position trigger signal and the pressure recording for a measurement angle of 0° .

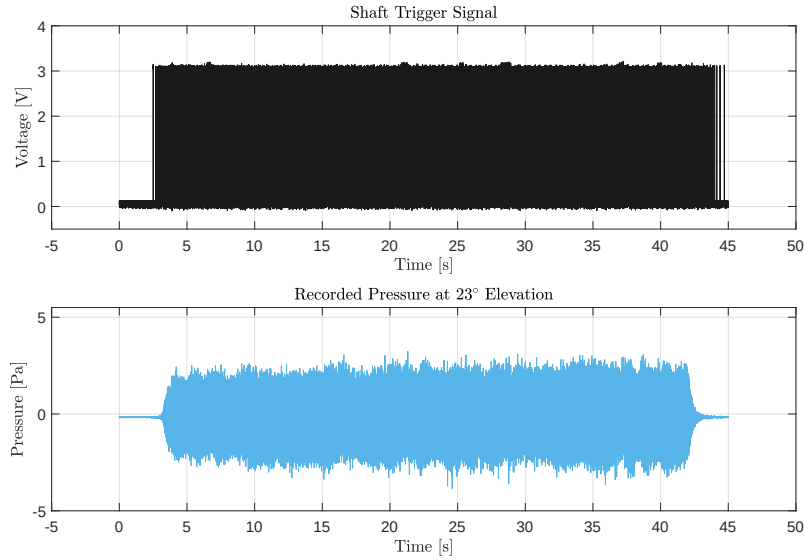


Figure 1: Trigger signal and a pressure recording at 0° .

Spectrogram of Recording at the 23° Recording Angle

Figure 2 shows the spectrogram for the Channel 1 pressure recording at an angle of 23° .

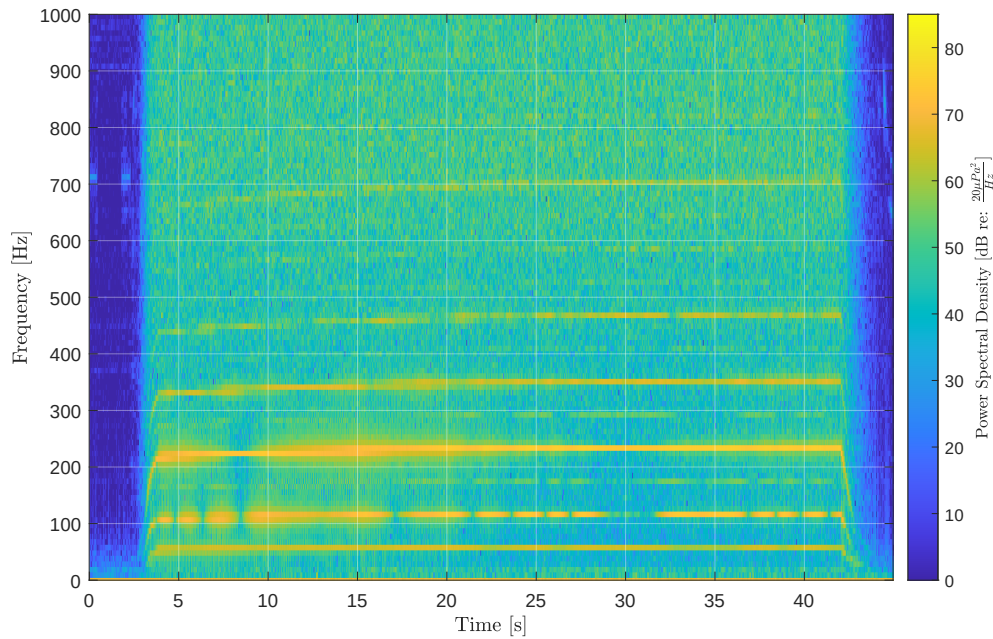


Figure 2: Spectrogram for the pressure recording at 23° .

The bottom trace in the spectrogram, which corresponds to the shaft rotation, is about 58.6 cycles per second.

Estimate of Shaft Rotation Speed

Figure 3 shows the estimated rotation speed of the shaft.

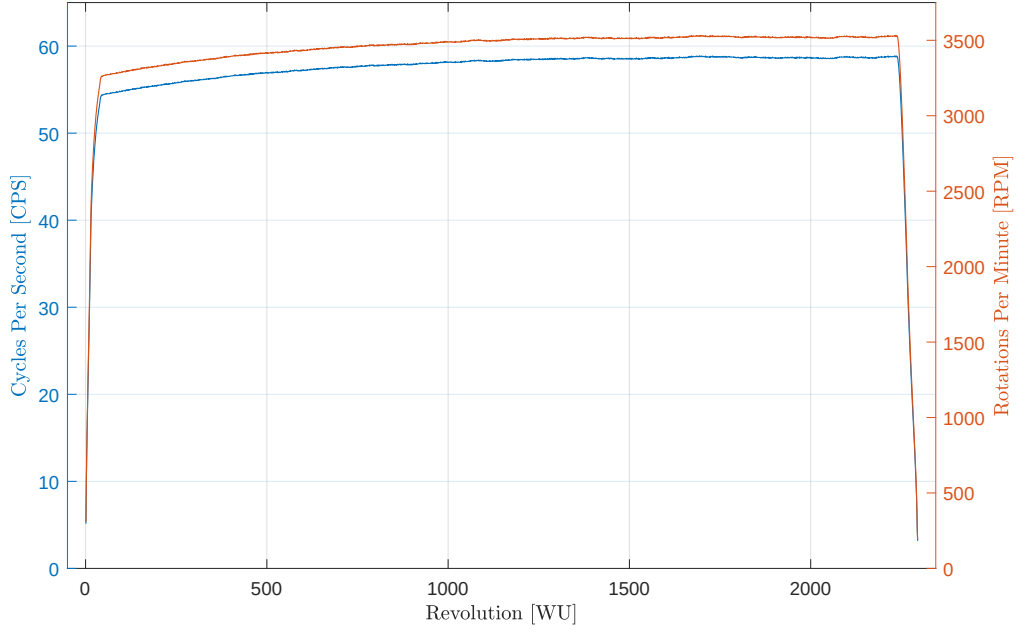


Figure 3: Rotation speed estimation.

The maximum shaft rotation speed is approximately 58.9 cycles per second, or 3,532 RPM. This maximum rotation speed occurs in revolution 1,672.

Second-order Tracking for the Channel 1 Pressure Recording

Figure 4 shows the second-order tracking profile for the Channel 1 pressure recording.

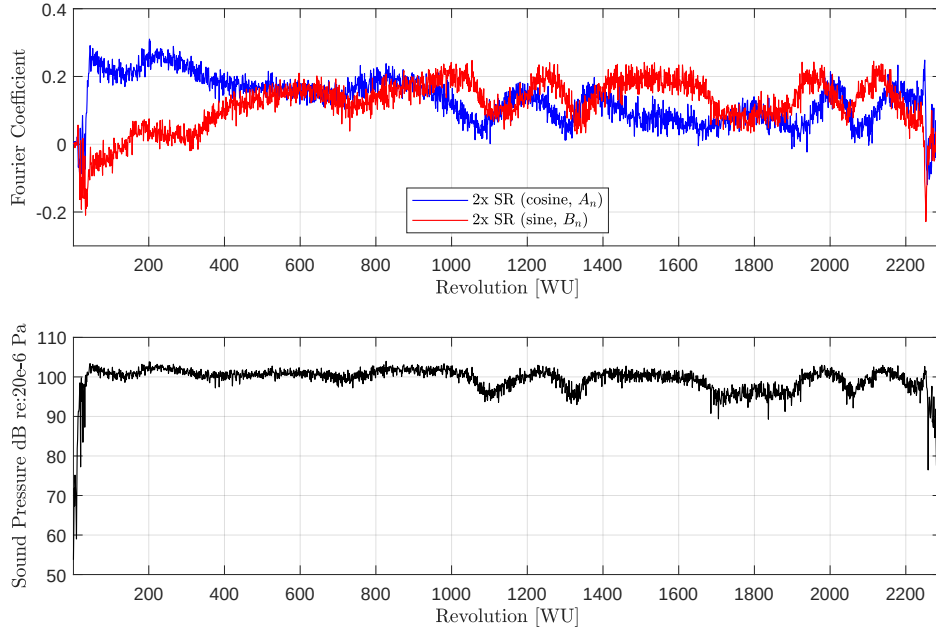


Figure 4: Rotation speed estimation.

The maximum shaft rotation speed is approximately 58.9 cycles per second, or 3,532 RPM. This maximum rotation speed occurs in revolution 1,672.

1 Appendix - Matlab Code

```
1
2
3
4 %% Synopsis
5
6 % Problem 5 — Order Tracking of Mobius Propeller Data
7
8 % See Lecture 26
9 % "D:\15 Downloads\00 GitHub\ACS_547\35 Lectures\26 Monday, April 21, 2025\Lecture 26 — Shafting and
   bearings.pptx"
10
11
12
13 %% Environment
14
15 close all; clear; clc;
16 % restoredefaultpath;
17
18 addpath( genpath( './00 Support' ), '-begin' );
19
20 set( groot, 'DefaultFigurePosition', [ 230 730 3*750 500 ] ); % x, y, width, height
21
22 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
23 set( 0, 'DefaultFigureWindowStyle', 'normal' );
24 set( 0, 'DefaultLineLineWidth', 0.6 );
25 set( 0, 'DefaultTextInterpreter', 'Latex' );
26
27 format ShortG;
28
29 color_map = slanCM( 'ColorBlind', 4 );
30
31 pause( 1 );
32
33
34
35 %% Load Data
36
37 load( 'mobius_prop_data.mat' ); % Variable(s): fs; mic_angles_degrees; p; trigger
38
39 % fs — Sample rate of 80 kHz (scalar).
40 % mic_angles_degrees — Microphone angles relative to horizontal plane (11-by-1; 23, 18, 6, 0, -6, -12,
   -18, -23, -28, and -32).
41 % p — Recorded pressures for the corresponding microphone angles (3,600,000-by-11).
42 % trigger — Shaft rotation synchronization signal (3,600,000-by-11).
43
44
45
46 %% Visualize Experimental Data
47
48 net_time = size( p, 1 ) / fs; % 45 seconds
49
50 time_indices = ( 0:1:( numel( trigger ) - 1 ) ) ./ fs;
51 time_indices = time_indices(:);
52
53
54 figure( 'Name', 'Trigger signal and Channel 1 Pressure Data' ); ...
55
56 h1 = subplot( 2, 1, 1 ); ...
57 plot( time_indices, trigger, 'Color', color_map( 1, : ) ); grid on;
58 xlabel( 'Time [s]' ); ylabel( 'Voltage [V]' ); title( 'Shaft Trigger Signal' );
59 ylim( [ -0.5 4 ] );
60 h2 = subplot( 2, 1, 2 ); ...
61 plot( time_indices, p( :, 1 ), 'Color', color_map( 2, : ) ); grid on;
62 xlabel( 'Time [s]' ); ylabel( 'Pressure [Pa]' ); title( 'Recorded Pressure at 23$\circ$
   Elevation' );
63 ylim( [ -5 5.5 ] );
64
65 linkaxes( [ h1 h2 ], 'x' );
66 xlim( [ -5 50 ] );
67
68
69
70 %% Spectrogram
71
72 signal = p(:, 1);
73
74 frame_length = 8192;
75 frame_hop = floor( 0.20 * frame_length );
76
77 a_spectrogram = spectrogram_September_26_2023( signal, frame_length, frame_hop, fs, 0 );
78
79 figure( 'Name', 'Spectrogram of Channel 1 Pressure Signal' ); ...
80
81 imagesc( a_spectrogram.time_indices, a_spectrogram.Sxx.frequencies, 20*log10( sqrt(a_spectrogram.
   Sxx.spectrum)./20e-6 ), [ 0 85 ] );
82 labelColorbar( 'Power Spectral Density [dB re: $\frac{20}{\mu}$ Pa$^2$]{Hz}$' ); grid on;
83 colormap parula; % Option: Turbo
84 set( gca, 'GridColor', 'w', 'GridAlpha', 0.4 );
85 xlabel( 'Time [s]' ); ylabel( 'Frequency [Hz]' );
```

```

86     set( gca, 'ydir', 'normal' );
87     ylim( [ 0 1e3 ] );
88
89
90
91 %% Locate the Leading Edges of the Trigger Data
92
93 % Smooth data to reduce noise; supports finding the leading edges.
94 smoothed_trigger_data = [ 0; diff( smooth( trigger, 'moving', 5 ) ) ];
95
96 % Threshold smoothed data to located leading edges.
97 threshold = 0.5;
98 edge_indices = find( threshold < smoothed_trigger_data );
99 leading_edges = edge_indices( 1:2:end );
100
101 temp = diff( leading_edges );
102 [ indices_to_remove, indices ] = find( temp == 2 );
103 clear temp;
104
105 temp = leading_edges;
106 temp( indices_to_remove ) = [ ];
107
108 leading_edges = temp;
109 clear temp;
110
111 time_indices_leading_edge = leading_edges ./ fs;
112
113
114 % figure( 'Name', 'Revolution Segmentation' ); ...
115 %
116 %     plot( trigger( 1:1:end ), 'Color', 'k' ); hold on;
117 %
118 %     for index = 1:1:numel( leading_edges )
119 %         line( [ leading_edges( index ) leading_edges( index ) ], [ 0 4 ], 'Color', 'b' ); grid on;
120 %     end
121 %
122 %     axis( [ 1 numel( trigger ) -0.5 4.5 ] );
123
124
125
126 %% Segment Microphone Recording Channels
127
128 % Channel 1: 23 degrees
129 % Channel 2: 18 degrees
130 % Channel 3: 12 degrees
131 % Channel 4: 6 degrees
132 % Channel 5: 0 degrees
133 % Channel 6: -6 degrees
134 % Channel 7: -12 degrees
135 % Channel 8: -18 degrees
136 % Channel 9: -23 degrees
137 % Channel 10: -28 degrees
138 % Channel 11: -32 degrees
139
140
141 start_indices = 1:1:( numel( leading_edges ) - 1 ); end_indices = 2:1:numel( leading_edges );
142
143 frame_set_indices = [ leading_edges( start_indices(:) ) leading_edges( end_indices(:) ) ];
144 number_of_revolutions = size( frame_set_indices, 1 );
145
146
147 for channel_index = 1:1:size( p, 2 )
148
149     for frame_index = 1:1:size( frame_set_indices, 1 )
150
151         channel_segments{ frame_index, channel_index } = p( frame_set_indices( frame_index, 1 ):1:
152             frame_set_indices( frame_index, 2 ), channel_index );
153
154         slope_theta = (2*pi) ./ size( channel_segments{ frame_index, channel_index }, 1 );
155         theta{ frame_index, channel_index } = slope_theta .* ( 0:1:size( channel_segments{
156             frame_index, channel_index }, 1 ) - 1 );
157
158     end
159 end
160
161
162 %% Compute Instantaneous RPM
163
164 for channel_index = 1:1:size( p, 2 )
165     for frame_index = 1:1:size( frame_set_indices, 1 )
166         rpm_estimate{ frame_index, channel_index } = 60 * ( 1 / ( numel( channel_segments{ frame_index,
167             channel_index } ) ./ fs ) );
168     end
169 end
170
171 figure( 'Name', 'Shaft Speed' ); ...
172
173 yyaxis left; ...
174 plot( [ rpm_estimate{ :, 1 } ] ./ 60 ); grid on;

```

```

175     ylabel( 'Cycles Per Second [CPS]' );
176     ylim( [ 0 65 ] );
177
178     yyaxis right; ...
179     plot( [ rpm_estimate{ :, 1 } ] ); grid on;
180     ylabel( 'Rotations Per Minute [RPM]' );
181
182     xlabel( 'Revolution [WU]' );
183
184     axis( [ -50 number_of_revolutions+50 0 3.75e3 ] );
185     shg;
186
187
188
189 %% Compute Fourier Coefficients
190
191 An = nan( size( frame_set_indices, 1 ), 11 ); Bn = An;
192
193
194 TRACKING_ORDER = 2;
195
196 for channel_index = 1:size( p, 2 )
197
198     for frame_index = 1:size( frame_set_indices, 1 )
199
200         M = numel( channel_segments{ frame_index, channel_index } );
201
202         An( frame_index, channel_index ) = ...
203             2/M * channel_segments{ frame_index, channel_index }.' * cos( TRACKING_ORDER/2 .* theta{
204                 frame_index, channel_index } ).';
205
206         Bn( frame_index, channel_index ) = ...
207             2/M * channel_segments{ frame_index, channel_index }.' * sin( TRACKING_ORDER/2 .* theta{
208                 frame_index, channel_index } ).';
209
210     end
211 end
212
213
214 %% Plot Fourier Coefficients An and Bn and the Overall Sound Pressure Level
215
216 figure( 'Name', 'Second-order Tracking and Associated Sound Pressure Level' ); ...
217
218 h1 = subplot( 2, 1, 1 ); ...
219 plot( An( :, 1 ), 'Color', 'b' ); hold on;
220 plot( Bn( :, 1 ), 'Color', 'r' ); grid on;
221 legend( '2x SR (cosine, $A_n$)', '2x SR (sine, $B_n$)', 'Location', 'South', 'Interpreter',
222         'Latex' );
223 xlabel( 'Revolution [WU]' ); ylabel( 'Fourier Coefficient' );
224 grid on; axis( [ 1 number_of_revolutions -0.3 0.4 ] );
225
226 h2 = subplot( 2, 1, 2 ); ...
227 plot( 10*log10( ( An( :, 1 ).^2 + Bn( :, 1 ).^2 ) ./ 2e-6^2 ), 'Color', 'k' ); grid on;
228 xlabel( 'Revolution [WU]' ); ylabel( 'Sound Pressure dB re:20e-6 Pa' );
229 axis( [ 1 number_of_revolutions 50 110 ] );
230
231 shg;
232
233
234 %% Clean-up
235
236 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
237     monitors = get( 0, 'MonitorPositions' );
238     if ( size( monitors, 1 ) == 1 )
239         autoArrangeFigures( 3, 4, 1 );
240     elseif ( 1 < size( monitors, 1 ) )
241         autoArrangeFigures( 2, 2, 2 );
242     end
243 end
244
245 fprintf( 1, '\n\n*** Processing Complete ***\n\n' );
246
247
248
249 %% Reference(s)
250
251 % https://www.mathworks.com/help/signal/ref/ordertrack.html#bvaw5ra-1-x
252
253 % https://www.mathworks.com/matlabcentral/answers/1439884-evaluate-rising-edge-sample-of-a-signal

```